

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ ИМЕНИ
ПАТРИСА ЛУМУМБЫ»
Факультет физико-математических и естественных наук
Кафедра математического моделирования и искусственного интеллекта

ОТЧЁТ

по лабораторной работе №7

Лабораторная работа 7. Дискретно-событийное моделирование
по дисциплине «Имитационное моделирование»

Выполнила: Исаева Зарина Исмаилбековна
Группа: НКНбд–01–23
Преподаватель: Кулябов Дмитрий Сергеевич, преподаватель дисциплины

Москва, 2026

Содержание

Цель работы	3
Постановка задачи	3
Теоретические сведения	3
Исходные параметры и организация эксперимента	3
Ход выполнения лабораторной работы	3
Результаты моделирования	4
Анализ и интерпретация результатов	5
Выводы	5
Листинг программы	6
Список литературы	7

Цель работы

Смоделировать систему массового обслуживания и систему с резервом и ремонтом в событийном представлении.

Постановка задачи

В рамках лабораторной работы необходимо:

- смоделировать систему массового обслуживания М/М/с;
- исследовать поведение модели с резервом и ремонтом по Россу;
- получить временные ряды длины очереди и загрузки каналов;
- оценить, как изменение резерва влияет на среднее время до отказа.

Теоретические сведения

Дискретно-событийное моделирование особенно эффективно для систем обслуживания, где важны моменты поступления заявок, начала и завершения обслуживания, а также переходы оборудования между состояниями исправности и отказа. Модель М/М/с является базовой постановкой для анализа очередей и использования параллельных каналов. Модель Росса с резервом и ремонтом описывает систему надёжности, в которой отказы и восстановление происходят случайно, а наличие резерва меняет среднее время до полного отказа. Такая постановка демонстрирует, как событийная симуляция помогает анализировать эксплуатационные сценарии.

Исходные параметры и организация эксперимента

- в системе М/М/с задаются интенсивности входного потока, обслуживания и число каналов;
- по времени фиксируются длина очереди и число занятых серверов;
- во второй части моделируется система с резервом и ремонтом;
- серия прогонов выполняется для разных размеров резерва.

Ход выполнения лабораторной работы

1. Реализован событийный механизм обработки заявок для системы М/М/с.
2. Сохранены временные ряды очереди и загрузки каналов.
3. Построена отдельная событийная модель с резервом и ремонтом.
4. Для нескольких размеров резерва рассчитано среднее время до отказа.
5. Подготовлены визуализации и сводные таблицы для двух сценариев исследования.

Результаты моделирования

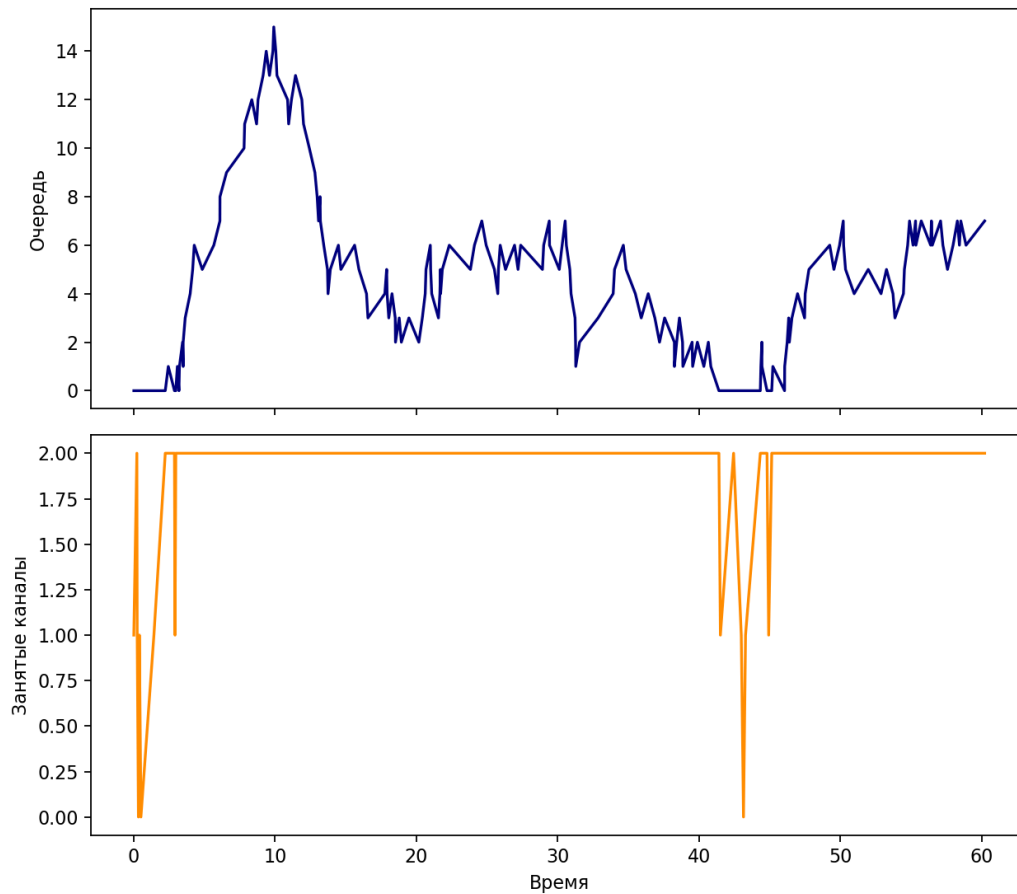


Рисунок 1: Очередь и загрузка в системе $M/M/c$

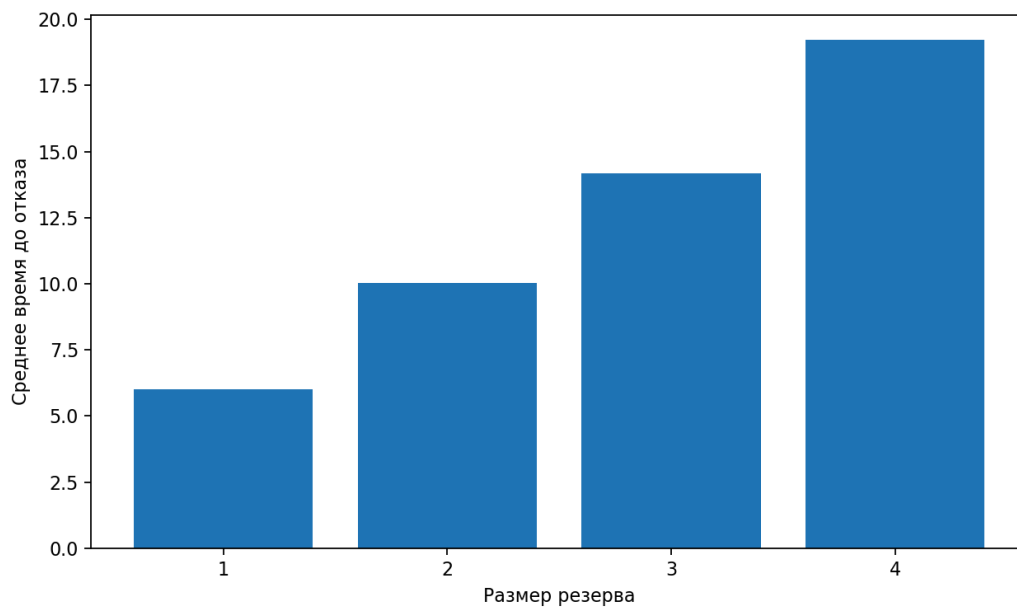


Рисунок 2: Среднее время до отказа в модели Росса

time	queue_length	busy_servers
0.019	0	1
0.237	0	2
0.259	0	1
0.346	0	0
0.411	0	1
0.523	0	0
1.441	0	1
2.245	0	2

Таблица 1: Фрагмент временного ряда системы М/М/с

reserve	mean_crash_time
1	6
2	10.035
3	14.194
4	19.216

Таблица 2: Таблица основных результатов

Анализ и интерпретация результатов

- Динамика очереди и загрузки каналов показывает изменение степени насыщения системы обслуживания во времени.
- Рост размера резерва увеличивает среднее время до отказа, поскольку система дольше сохраняет работоспособность при случайных сбоях.
- Совместное рассмотрение двух моделей демонстрирует универсальность событийного подхода для задач обслуживания и надёжности.

Выводы

1. Построены две дискретно-событийные модели: М/М/с и модель Росса.
2. Получены количественные оценки очереди, загрузки и надёжности системы.
3. Показано, что резерв и ремонт являются эффективными механизмами увеличения времени безотказной работы.

Листинг программы

Этот файл сгенерирован автоматически из qmd-источника.

```
using Random
using Printf
using Statistics

results_dir = normpath(joinpath(@__DIR__, "..", "results", "data"))
mkpath(results_dir)
Random.seed!(19)

function write_csv(path, headers, rows)
    open(path, "w") do io
        println(io, join(headers, ","))
        for row in rows
            println(io, join(string.(row), ","))
        end
    end
end

exp_time(rate) = -log(rand()) / rate

function simulate_mmc(; lambda = 1.4, mu = 0.9, c = 2, t_max = 60.0)
    t = 0.0
    next_arrival = exp_time(lambda)
    departures = Float64[]
    queue = 0
    busy = 0
    rows = Vector{Vector{String}}()
    while t < t_max
        next_departure = isempty(departures) ? Inf : minimum(departures)
        if next_arrival < next_departure
            t = next_arrival
            next_arrival += exp_time(lambda)
            if busy < c
                busy += 1
                push!(departures, t + exp_time(mu))
            else
                queue += 1
            end
        else
            t = next_departure
            idx = argmin(departures)
            deleteat!(departures, idx)
            if queue > 0
                queue -= 1
                push!(departures, t + exp_time(mu))
            else
                busy -= 1
            end
        end
        push!(rows, [@sprintf("%.4f", t), string(queue), string(busy)])
    end
    return rows
end

function simulate_ross_once(; reserve = 2, running = 6, failure_rate = 0.07,
```

```

repair_rate = 0.22)
    t = 0.0
    reserve_now = reserve
    repair_queue = 0
    repair_busy = false
    next_repair = Inf
    while true
        next_failure = t + exp_time(failure_rate * running)
        if next_failure < next_repair
            t = next_failure
            if reserve_now > 0
                reserve_now -= 1
                repair_queue += 1
                if !repair_busy
                    repair_busy = true
                    repair_queue -= 1
                    next_repair = t + exp_time(repair_rate)
                end
            end
            else
                return t
            end
        else
            t = next_repair
            reserve_now += 1
            if repair_queue > 0
                repair_queue -= 1
                next_repair = t + exp_time(repair_rate)
            else
                repair_busy = false
                next_repair = Inf
            end
        end
    end
end

mmc_rows = simulate_mmc()
write_csv(joinpath(results_dir, "mmc_timeseries.csv"), ["time", "queue_length",
"busy_servers"], mmc_rows)

ross_rows = Vector{Vector{String}}{()}
for reserve in 1:4
    samples = [simulate_ross_once(reserve = reserve) for _ in 1:120]
    push!(ross_rows, [string(reserve), @sprintf("%.4f", mean(samples))])
end
write_csv(joinpath(results_dir, "ross_summary.csv"), ["reserve", "mean_crash_time"],
ross_rows)
println("lab07 done")

```

Список литературы

- Jerry Banks and John Carson and Barry Nelson and David Nicol. Discrete-Event System Simulation. Prentice Hall, 2010.
- Sheldon M. Ross. Introduction to Probability Models. Academic Press, 2014.